

PUC RIO

Pontifícia Universidade Católica do Rio de Janeiro
Pós Graduação Lato Sensu em Ciência de Dados e Analytics

Sprint: Engenharia de Dados

Construção de um Pipeline de Dados na Nuvem

Análise de Dados de Empresa Fictícia

Aluno: Diego Castanheira Chester Franco

Matrícula: 4052025000131

Objetivo geral:

A partir de um conjunto de dados fictício, criado por cientistas de dados da IBM, o objetivo deste projeto é identificar e analisar os fatores que contribuem para a rotatividade de funcionários. A análise busca explorar questões relevantes sobre o perfil e as condições de trabalho dos colaboradores, incluindo a taxa geral de rotatividade, sua distribuição por área, cargo e gênero, a distância entre a residência e o trabalho por cargo e situação de desligamento, e a comparação da renda média mensal segundo o nível de escolaridade e a ocorrência de rotatividade. Também serão avaliadas possíveis associações entre desligamento, horas extras, viagens a trabalho, satisfação, equilíbrio entre vida pessoal e profissional, idade e tempo de empresa. Com isso, pretende-se compreender quais características pessoais, profissionais e organizacionais estão mais associadas à saída de funcionários e fornecer subsídios para a tomada de decisões voltadas à retenção de pessoas.

Problema a ser estudado: rotatividade de pessoas.

Perguntas Gerais:

1. Qual é a taxa geral de rotatividade de funcionários?
2. Quais áreas e cargos concentram as maiores taxas de rotatividade?
3. Como a distância entre a residência e o trabalho varia por cargo e situação de desligamento?
4. Horas extras, viagens a trabalho, satisfação e equilíbrio entre vida pessoal e profissional estão associados à saída dos funcionários?
5. Como renda mensal, nível de escolaridade, idade e tempo de empresa diferem entre funcionários desligados e retidos?
6. A taxa de rotatividade e seus fatores associados apresentam diferenças entre os gêneros?

Coleta dos Dados

Os dados utilizados neste projeto foram obtidos em a partir do conjunto fictício *IBM HR Analytics Employee Attrition & Performance*, criado por cientistas de dados da IBM e disponibilizado na plataforma Kaggle. O conjunto foi desenvolvido para investigar os fatores relacionados à rotatividade de funcionários e permite análises como a distribuição da distância entre residência e trabalho por cargo e situação de desligamento, além da comparação da renda média mensal por nível de escolaridade e ocorrência de rotatividade. Os dados podem ser consultados em [IBM HR Analytics Employee Attrition & Performance](https://www.kaggle.com/datasets/pavansubhasht/ibm-hr-analytics-attrition-dataset) (<https://www.kaggle.com/datasets/pavansubhasht/ibm-hr-analytics-attrition-dataset>).

O *dataset* é fornecido em um arquivo de valores separados por vírgula (CSV) e contém 1.470 registros de funcionários e 35 variáveis. Entre os atributos disponíveis estão idade, gênero, área, cargo, distância de casa, escolaridade, renda mensal, horas extras, frequência de viagens, satisfação no trabalho, equilíbrio entre vida pessoal e profissional e tempo de empresa, além da variável de rotatividade (*Attrition*), que identifica se ocorreu desligamento.

Arquitetura do Pipeline de Dados

O pipeline de dados deste projeto foi organizado conforme a arquitetura em camadas Bronze, Silver e Gold, permitindo controlar as etapas de ingestão, tratamento, transformação e disponibilização das informações.

A camada Bronze armazena os dados brutos importados do arquivo original `WA_Fn-UseC_HR-Employee-Attrition.csv`, pertencente ao conjunto IBM HR Analytics Employee Attrition & Performance. Nessa etapa, os dados são preservados conforme a fonte, sem aplicação de regras de negócio, limpeza ou padronização.

Na camada Silver, os dados passam por processos de validação e tratamento, como conferência dos tipos das colunas, identificação de valores ausentes e registros duplicados, correção de inconsistências, análise de valores extremos e padronização das variáveis categóricas. Como resultado, essa camada disponibiliza dados tratados e confiáveis para a construção do modelo analítico.

A camada Gold será estruturada por meio de um esquema estrela, composto por uma tabela fato central e tabelas dimensão relacionadas. A tabela fato concentrará os indicadores e as medidas associados à permanência e ao desligamento dos funcionários, enquanto as dimensões organizarão informações descritivas, como características do colaborador, cargo, departamento, escolaridade, condições de trabalho e tempo de empresa.

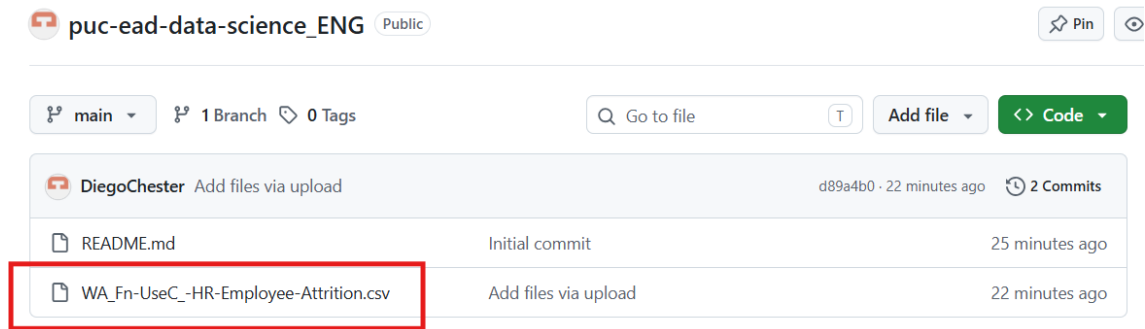
Essa estrutura permitirá calcular e analisar métricas como taxa de rotatividade, quantidade de funcionários desligados e retidos, renda média, distância média entre residência e trabalho e comparações por gênero, departamento, cargo, escolaridade e condições de trabalho. O esquema estrela também facilitará a realização de consultas analíticas e a aplicação de filtros por diferentes perspectivas.

A organização do pipeline em camadas, combinada com a modelagem dimensional em esquema estrela, proporciona maior rastreabilidade, governança, qualidade e reprodutibilidade, além de tornar mais claro o fluxo de transformação dos dados desde o arquivo original até sua utilização nas análises.

Carga dos Dados

O repositório inicial do arquivo original para o MVP, bem como a documentação de todo o projeto está no GitHub. A construção do pipeline de carga e tratamento de dados foi realizada no Databricks Free Edition .

Passo 1: Carga do arquivo original no GitHub:



Passo 2: copiar o link RAW do arquivo original

link: https://raw.githubusercontent.com/DiegoChester/puc-ead-data-science_ENG/refs/heads/main/WA_Fn-UseC_-HR-Employee-Attrition.csv

Passo 3: fazer a ingestão, dar carga no Databricks Free Edition (Bronze: ingestão + criação da tabela Delta)

No Workspace, foi criado um Notebook com nome `pipeline_carga_bronze`, onde recebe a carga inicial do arquivo original que está no GitHub. Essa etapa bronze foi detalhada a seguir.

Na etapa Bronze, ao realizar a ingestão dos dados exatamente como eles se encontram na fonte original, preservando sua estrutura e conteúdo sem qualquer transformação, encontrou-se dificuldades devido, os dados de RH (Employee Attrition) estarem disponibilizados em um arquivo CSV hospedado no GitHub.

Inicialmente, tentou-se realizar a leitura direta desse arquivo utilizando o Spark, porém o ambiente Databricks Free Edition não oferece suporte à leitura de arquivos via HTTPS, resultando em erros relacionados ao sistema de arquivos (UNSUPPORTED_FILE_SYSTEM).

Para contornar essa limitação, adotou-se uma estratégia alternativa baseada em três passos: download do arquivo via Python, leitura com pandas e conversão para DataFrame Spark. Essa abordagem garante a ingestão confiável dos dados, mantendo a compatibilidade com o ambiente e respeitando os princípios da camada Bronze.

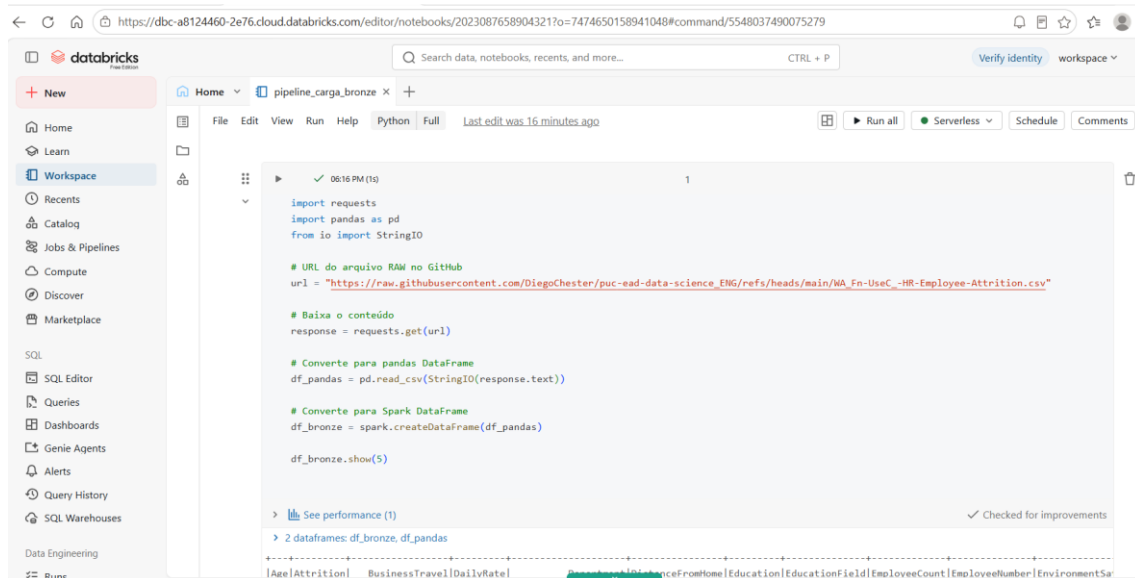
Primeiro, foi utilizada a biblioteca `requests` para realizar uma requisição HTTP ao endereço RAW do arquivo no GitHub. O conteúdo retornado pela requisição foi tratado como texto, representando integralmente o CSV original. Em seguida, esse texto foi convertido em um “arquivo virtual” em memória utilizando `StringIO`, permitindo que o pandas realizasse a leitura do CSV normalmente por meio da função `read_csv`. O resultado dessa operação foi um DataFrame pandas contendo os dados brutos.

Com o DataFrame pandas carregado, o próximo passo foi convertê-lo para um DataFrame Spark utilizando `spark.createDataFrame`. Essa conversão é essencial, pois o Spark DataFrame é o formato nativo utilizado nas camadas Bronze, Silver e Gold dentro da arquitetura Medalhão. Após a conversão, os dados foram exibidos para validação inicial, garantindo que a ingestão ocorreu corretamente.

Por fim, o DataFrame Bronze foi persistido como uma tabela Delta no catálogo do Databricks por meio do comando `saveAsTable`. Essa operação cria uma tabela física chamada `bronze_hr_attrition`, que passa a ser acessível pelo Catalog. A partir desse momento, a tabela Bronze está oficialmente registrada, podendo ser documentada, consultada e utilizada como base para as próximas camadas de processamento (Silver e Gold).

Essa abordagem assegura que os dados brutos foram ingeridos de forma íntegra, auditável e compatível com o ambiente, estabelecendo uma base sólida para as etapas subsequentes de limpeza, padronização e modelagem.

Tela da carga:



```
import requests
import pandas as pd
from io import StringIO

# URL do arquivo RAW no GitHub
url = "https://raw.githubusercontent.com/DiegoChester/puc-ead-data-science_ENG/refs/heads/main/NA_Fn-UseC_-HR-Employee-Attrition.csv"

# Baixa o conteúdo
response = requests.get(url)

# Converte para pandas DataFrame
df_pandas = pd.read_csv(StringIO(response.text))

# Converte para Spark DataFrame
df_bronze = spark.createDataFrame(df_pandas)

df_bronze.show(5)
```

> 2 dataframes: df_bronze, df_pandas

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EmployeeCount	EmployeeNumber	EnvironmentSa
41	Yes	Travel_Rarely	1102	Sales	1	2	Life Sciences	1	1		
49	No	Travel_Frequently	279	Research & Develo...	8	1	Life Sciences	1	2		
37	Yes	Travel_Rarely	1373	Research & Develo...	2	2	Other	1	4		
33	No	Travel_Frequently	1392	Research & Develo...	3	4	Life Sciences	1	5		
27	No	Travel_Rarely	591	Research & Develo...	2	1	Medical	1	7		

only showing top 5 rows

Resultado da Carga: mostrando as 5 primeiras linhas com `df_bronze.show(5)`

df_bronze.show(5)

> [See performance \(1\)](#)

✓ Checked for improvements

> 2 dataframes: df_bronze, df_pandas

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EmployeeCount	EmployeeNumber	EnvironmentSa
41	Yes	Travel_Rarely	1102	Sales	1	2	Life Sciences	1	1		
49	No	Travel_Frequently	279	Research & Develo...	8	1	Life Sciences	1	2		
37	Yes	Travel_Rarely	1373	Research & Develo...	2	2	Other	1	4		
33	No	Travel_Frequently	1392	Research & Develo...	3	4	Life Sciences	1	5		
27	No	Travel_Rarely	591	Research & Develo...	2	1	Medical	1	7		

only showing top 5 rows

Criação da tabela Delta na camada Bronze dentro da Arquitetura Medalhão:

```
06:17 PM (20s) 2
df_bronze.write.format("delta").mode("overwrite").saveAsTable("bronze_hr_attrition")

See performance (1) Checked for improvements
```

O código acima é o responsável por persistir os dados brutos que foram carregados e convertidos para Spark em uma tabela Delta dentro do Databricks. Essa etapa transforma o DataFrame em uma entidade física e catalogada, permitindo que ele seja consultado, versionado, auditado e utilizado pelas camadas Silver e Gold.

Ao salvar o DataFrame como tabela Delta, garantimos que os dados brutos ficam armazenados de forma estruturada, com suporte a transações ACID, histórico de versões e otimizações internas do Delta Lake. O uso do modo overwrite assegura que, caso a tabela já exista, ela será substituída integralmente, mantendo a camada Bronze sempre sincronizada com a fonte original. A função saveAsTable registra a tabela no catálogo do Databricks, tornando-a acessível via SQL, notebooks e ferramentas de governança.

Resumo da Etapa Bronze:

O notebook da camada Bronze (pipeline_carga_bronze) contém apenas dois blocos de código:

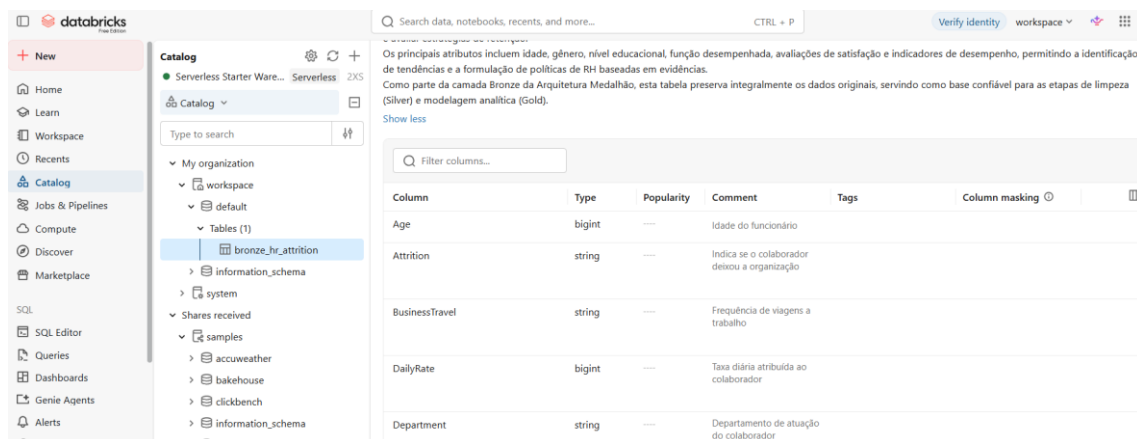
- (1) ingestão dos dados via requisição HTTP, leitura com pandas e conversão para Spark;
- (2) persistência do DataFrame Bronze como tabela Delta no catálogo do Databricks.

Todos os blocos de tentativa que resultaram em erro foram removidos, mantendo o pipeline limpo, organizado e alinhado às boas práticas da Arquitetura Medalhão.

Passo 4: Catalogação da Etapa Bronze

Em Catalog > My organization > workspace > default > bronze_hr_attrition

O tipo de dados vem automático e a descrição de cada coluna coloca-se no Comment:



Column	Type	Popularity	Comment	Tags	Column masking
Age	bigint	—	Idade do funcionário		
Attrition	string	—	Indica se o colaborador deixou a organização		
BusinessTravel	string	—	Frequência de viagens a trabalho		
DailyRate	bigint	—	Taxa diária atribuída ao colaborador		
Department	string	—	Departamento de atuação do colaborador		

Abaixo, foi elaborada a tabela com mais detalhes do catálogo de dados da Etapa Bronze:

Foi levantado os dados conforme o arquivo original, verificado os tipos de dados, descrição, unidade, valor mínimo, valor máximo e categoria, conforme a tabela a seguir.

Tabela 1 - Catálogo de Dados - Bronze

Nome da Coluna	Tipo do Dado	Descrição	Unidade	Valor Mínimo	Valor Máximo	Categoria
Age	int	Idade do colaborador	Anos	18	60	N/A
Attrition	string	Indica se o colaborador deixou a organização	N/A	N/A	N/A	“Yes”, “No”
BusinessTravel	string	Frequência de viagens a trabalho	N/A	N/A	N/A	“Travel_Rarely”, “Travel_Frequently”, “Non-Travel”
DailyRate	int	Taxa diária atribuída ao colaborador	Unidade monetária/dia	102	1499	N/A
Department	string	Departamento de atuação do colaborador	N/A	N/A	N/A	“Sales”, “Research & Development”, “Human Resources”
DistanceFromHome	int	Distância entre residência e trabalho	Unidade de distância	1	29	N/A
Education	int	Nível educacional codificado de 1 a 5	Escala 1–5	1	5	1=“Below College”; 2=“College”; 3=“Bachelor”; 4=“Master”; 5=“Doctor”
EducationField	string	Área de formação	N/A	N/A	N/A	“Life Sciences”, “Other”, “Medical”, “Marketing”, “Technical Degree”, “Human Resources”
EmployeeCount	int	Contador técnico constante por registro	Contagem	1	1	N/A
EmployeeNumber	int	Identificador único do colaborador	Identificador	1	2068	N/A
EnvironmentSatisfaction	int	Satisfação com o ambiente, escala de 1 a 4	Escala 1–4	1	4	1=“Low”; 2=“Medium”; 3=“High”; 4=“Very High”
Gender	string	Gênero informado	N/A	N/A	N/A	“Female”, “Male”
HourlyRate	int	Taxa horária atribuída ao colaborador	Unidade monetária/hora	30	100	N/A
JobInvolvement	int	Envolvimento com o trabalho, escala de 1 a 4	Escala 1–4	1	4	1=“Low”; 2=“Medium”; 3=“High”; 4=“Very High”

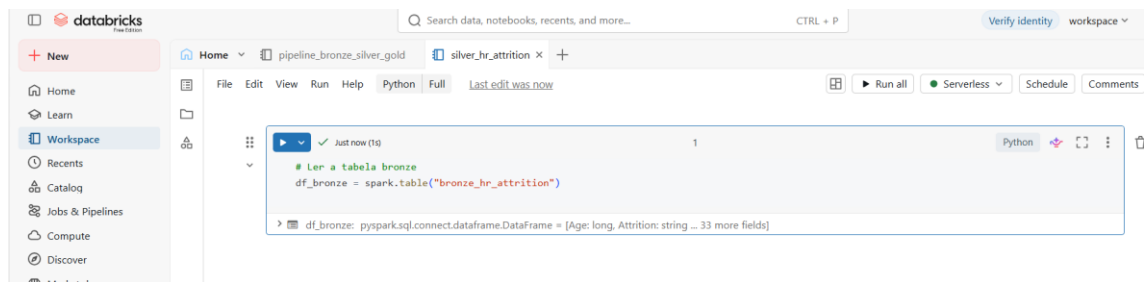
JobLevel	int	Nível hierárquico, escala de 1 a 5	Escala 1–5	1	5	1; 2; 3; 4; 5
JobRole	string	Função/cargo exercido	N/A	N/A	N/A	“Sales Executive”, “Research Scientist”, “Laboratory Technician”, “Manufacturing Director”, “Healthcare Representative”, “Manager”, “Sales Representative”, “Research Director”, “Human Resources”
JobSatisfaction	int	Satisfação com o trabalho, escala de 1 a 4	Escala 1–4	1	4	1=“Low”; 2=“Medium”; 3=“High”; 4=“Very High”
MaritalStatus	string	Estado civil	N/A	N/A	N/A	“Single”, “Married”, “Divorced”
MonthlyIncome	int	Renda mensal	Unidade monetária/mês	1009	19999	N/A
MonthlyRate	int	Taxa mensal atribuída ao colaborador	Unidade monetária/mês	2094	26999	N/A
NumCompaniesWorked	int	Quantidade de empresas em que trabalhou	Contagem	0	9	N/A
Over18	string	Indicador de idade superior a 18 anos	N/A	N/A	N/A	“Y”
OverTime	string	Indica realização de horas extras	N/A	N/A	N/A	“Yes”, “No”
PercentSalaryHike	int	Percentual do último aumento salarial	Percentual	11	25	N/A
PerformanceRating	int	Avaliação de desempenho	Escala 3–4	3	4	3=“Excellent”; 4=“Outstanding”
RelationshipSatisfaction	int	Satisfação com relacionamentos, escala de 1 a 4	Escala 1–4	1	4	1=“Low”; 2=“Medium”; 3=“High”; 4=“Very High”
StandardHours	int	Carga horária padrão constante	Horas	80	80	N/A
StockOptionLevel	int	Nível de opção de ações, de 0 a 3	Escala 0–3	0	3	0; 1; 2; 3
TotalWorkingYears	int	Total de anos de experiência profissional	Anos	0	40	N/A
TrainingTimesLastYear	int	Treinamentos realizados no último ano	Contagem anual	0	6	N/A
WorkLifeBalance	int	Equilíbrio vida-trabalho, escala de 1 a 4	Escala 1–4	1	4	1=“Bad”; 2=“Good”; 3=“Excellent”; 4=“Outstanding”

						3="Better"; 4="Best"
YearsAtCompany	int	Anos na empresa atual	Anos	0	40	N/A
YearsInCurrentRole	int	Anos na função atual	Anos	0	18	N/A
YearsSinceLastPromotion	int	Anos desde a última promoção	Anos	0	15	N/A
YearsWithCurrManager	int	Anos com o gestor atual	Anos	0	17	N/A

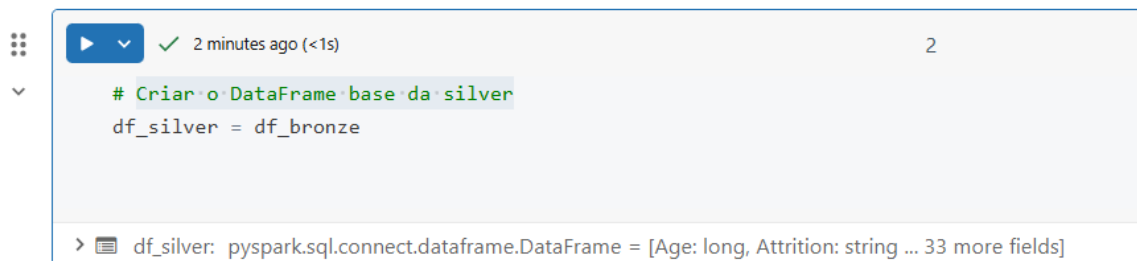
Ao preparar o catálogo de dados, foi observado que a coluna “PerformanceRating” contém dados referentes a 3 e 4, ou seja, não foi encontrado números 1 e 2. Sendo assim, o catálogo de dados está refletindo os dados brutos do arquivo original, mesmo sabendo que no futuro pode ser atualizado a instância e aparecer a classificação 1 ou 2. Para fins de registro, as escalas possíveis são: 1 “Low”; 2 “Good”; 3 “Excellent”; 4 “Outstanding”.

Posteriormente, analisaremos novamente o Catálogo de Dados, após estruturar o *pipeline* de dados, carga na nuvem e ETL dos dados.

Passo 5: Etapa Silver – ler tabela bronze



Passo 6: Etapa Silver – Criar o DataFrame base da silver



Passo 7: Etapa Silver – padronizar tipos

```
Just now (2s) 2

from pyspark.sql.functions import when, col

df_silver = (
    df_bronze
    # Age como inteiro
    .withColumn("Age", col("Age").cast("int"))

    # Attrition como boolean
    .withColumn("Attrition", when(col("Attrition") == "Yes", True).otherwise(False))

    # OverTime como boolean
    .withColumn("OverTime", when(col("OverTime") == "Yes", True).otherwise(False))

    # Income como double
    .withColumn("MonthlyIncome", col("MonthlyIncome").cast("double"))

    # PercentSalaryHike como double
    .withColumn("PercentSalaryHike", col("PercentSalaryHike").cast("double"))
)

> df_silver: pyspark.sql.connect.dataframe.DataFrame = [Age: integer, Attrition: boolean ... 33 more fields]
```

Passo 8: Etapa Silver – padronizar textos

```
3 minutes ago (<1s) 4

# padronizar textos
from pyspark.sql.functions import lower, trim

df_silver = (
    df_silver
    .withColumn("BusinessTravel", trim(lower(col("BusinessTravel"))))
    .withColumn("Department", trim(lower(col("Department"))))
    .withColumn("EducationField", trim(lower(col("EducationField"))))
    .withColumn("Gender", trim(lower(col("Gender"))))
)

> df_silver: pyspark.sql.connect.dataframe.DataFrame = [Age: integer, Attrition: boolean ... 33 more fields]
```

Passo 9: tratar valores nulos

```
08:50 PM (<1s) 5

# tratar valores nulos
df_silver = df_silver.fillna({
    "EnvironmentSatisfaction": 0,
    "JobSatisfaction": 0,
    "WorkLifeBalance": 0
})

> df_silver: pyspark.sql.connect.dataframe.DataFrame = [Age: integer, Attrition: boolean ... 33 more fields]
```

Passo 10: Renomear todas as colunas para snake_case

```
▶ 08:50 PM (<1s) 6

#Renomear todas as colunas para snake_case
import re

def to_snake_case(col_name):
    s1 = re.sub('(\.)([A-Z][a-z]+)', r'\1_2', col_name)
    s2 = re.sub('([a-z0-9])([A-Z])', r'\1_2', s1)
    return s2.lower()

for col_name in df_silver.columns:
    df_silver = df_silver.withColumnRenamed(col_name, to_snake_case(col_name))
```

Passo 11: Criar a tabela Silver

```
▶ 08:50 PM (3s) 7

#Criar a tabela Silver
df_silver.write.format("delta").mode("overwrite").saveAsTable("silver_hr_attrition")

> See performance \(1\)
```

Passo 12: Validar a tabela Silver

```
▶ 08:50 PM (1s)

#Validar a Silver
spark.table("silver_hr_attrition").display()

> See performance \(1\)
```

Resultado:

08:50 PM (1s) 8

```
spark.table("silver_hr_attrition").display()
```

> [See performance \(1\)](#) [Optimize](#)

	age	attrition	business_travel	daily_rate	department	distance_from_home	education	education
5	27	false	travel_rarely	591	research & developme...	2	1	medical
6	32	false	travel_frequently	1005	research & developme...	2	2	life scienc
7	59	false	travel_rarely	1324	research & developme...	3	3	medical
8	30	false	travel_rarely	1358	research & developme...	24	1	life scienc
9	38	false	travel_frequently	216	research & developme...	23	3	life scienc
10	36	false	travel_rarely	1299	research & developme...	27	3	medical
11	35	false	travel_rarely	809	research & developme...	16	3	medical
12	29	false	travel_rarely	153	research & developme...	15	2	life scienc
13	31	false	travel_rarely	670	research & developme...	26	1	life scienc
14	34	false	travel_rarely	1346	research & developme...	19	2	medical
15	28	true	travel_rarely	103	research & developme...	24	3	life scienc
16	29	false	travel_rarely	1389	research & developme...	21	4	life scienc
17	32	false	travel_rarely	334	research & developme...	5	2	life scienc

Passo 13: verificar o Catálogo de dados

No Databricks aparece automaticamente em Catalog:

databricks

Search data, notebooks, recents, and more... CTRL + P

New

- Home
- Learn
- Workspace
- Recents
- Catalog**
- Jobs & Pipelines
- Compute
- Discover
- Marketplace

Catalog

Serverless Starter Ware... Serverless 2XS

Catalog

Type to search

- My organization
 - workspace
 - default**
 - Tables (1)
 - information_sche...
 - system
 - Shares received
 - samples
 - accuweather
 - bakehouse
 - clickbench
 - information_schema

Catalog Explorer > workspace >

default

Overview Details Permissions Policies

Description

Default schema (auto-created)

Filter tables

Tables 2 Volumes 0 Models 0 Functions 0 Secrets 0 Services 0 Skills 0

Connections 0

Sort

Name	Owner	Created at	Popularity	Commer
bronze_hr_attrition	diegochester@g...	Sep 27, 2026, 06:...		Esta tabe
silver_hr_attrition	diegochester@g...	Sep 27, 2026, 08:...		

Column	Type	Popularity	Comment	Tags	Column masking ⓘ
age	int				
attrition	boolean				
business_travel	string				
daily_rate	bigint		Taxa diária atribuída ao colaborador		
department	string				
distance_from_home	bigint		Distância entre residência e trabalho		
education	bigint		Nível educacional codificado de 1 a 5		
education_field	string				
employee_count	bigint		Contador técnico constante por registro		
employee_number	bigint		Identificador único do		

Passo 14: Etapa GOLD

- carregar a Silver

```
# Bloco 1 - Carregar Silver
df_silver = spark.table("workspace.default.silver_hr_attrition")
df_silver.display()
```

> [See performance \(1\)](#) [Optimize](#)

> `df_silver: pyspark.sql.connect.dataframe.DataFrame = [age: integer, attrition: boolean ... 33 more fields]`

	age	attrition	business_travel	daily_rate	department	distance_from_home	education	education_level
1	41	true	travel_rarely	1102	sales	1	2	life science
2	49	false	travel_frequently	279	research & development	8	1	life science
3	37	true	travel_rarely	1373	research & development	2	2	other
4	33	false	travel_frequently	1392	research & development	3	4	life science
5	27	false	travel_rarely	591	research & development	2	1	medical
6	32	false	travel_frequently	1005	research & development	2	2	life science
7	59	false	travel_rarely	1324	research & development	3	3	medical
8	30	false	travel_rarely	1358	research & development	24	1	life science
9	38	false	travel_frequently	216	research & development	23	3	life science
10	36	false	travel_rarely	1299	research & development	27	3	medical

- Criar gold_attrition_metrics

```
Just now (6s) 2 Python

# Bloco 2 - Criar gold_attrition_metrics

from pyspark.sql import functions as F

df_gold_metrics = (
    df_silver
    .agg(
        F.count("*").alias("total_employees"),
        F.sum(F.col("attrition").cast("int")).alias("total_attrition"),
        (F.sum(F.col("attrition").cast("int")) / F.count("*")).alias("attrition_rate"),
        F.avg("monthly_income").alias("avg_monthly_income"),
        F.avg("age").alias("avg_age"),
        F.avg("years_at_company").alias("avg_years_at_company"),
        F.avg(F.col("over_time").cast("int")).alias("percent_overtime"),
        F.avg("job_satisfaction").alias("avg_job_satisfaction"),
        F.avg("work_life_balance").alias("avg_work_life_balance")
    )
)

df_gold_metrics.write.format("delta").mode("overwrite").saveAsTable("workspace.default.gold_attrition_metrics")

> See performance \(1\) ✓ Checked for improvements

df_gold_metrics: pyspark.sql.connect.dataframe.DataFrame = [total_employees: long, total_attrition: long ... 7 more fields]
```

- Criar gold_attrition_by_department

```
Just now (5s) 3 Python

# Bloco 3 - Criar gold_attrition_by_department

df_gold_dept = (
    df_silver
    .groupBy("department")
    .agg(
        F.count("*").alias("total_employees"),
        F.sum(F.col("attrition").cast("int")).alias("total_attrition"),
        (F.sum(F.col("attrition").cast("int")) / F.count("*")).alias("attrition_rate"),
        F.avg("monthly_income").alias("avg_monthly_income"),
        F.avg("age").alias("avg_age"),
        F.avg("job_satisfaction").alias("avg_job_satisfaction"),
        F.avg("years_at_company").alias("avg_years_at_company")
    )
)

df_gold_dept.write.format("delta").mode("overwrite").saveAsTable("workspace.default.gold_attrition_by_department")

> See performance \(1\) ✓ Checked for improvements

df_gold_dept: pyspark.sql.connect.dataframe.DataFrame = [department: string, total_employees: long ... 6 more fields]
```

- Criar gold_attrition_by_job_role

```
Just now (4s) 4 Python

# Bloco 4 - Criar gold_attrition_by_job_role

df_gold_role = (
    df_silver
    .groupBy("job_role")
    .agg(
        F.count("*").alias("total_employees"),
        F.sum(F.col("attrition").cast("int")).alias("total_attrition"),
        (F.sum(F.col("attrition").cast("int")) / F.count("*")).alias("attrition_rate"),
        F.avg("monthly_income").alias("avg_monthly_income"),
        F.avg("years_in_current_role").alias("avg_years_in_role"),
        F.avg("job_satisfaction").alias("avg_job_satisfaction")
    )
)

df_gold_role.write.format("delta").mode("overwrite").saveAsTable("workspace.default.gold_attrition_by_job_role")

> See performance \(1\) ✓ Checked for improvements

df_gold_role: pyspark.sql.connect.dataframe.DataFrame = [job_role: string, total_employees: long ... 5 more fields]
```

- Validar as tabelas Gold

```
# Bloco 5 - Validar as tabelas Gold

print("gold_attrition_metrics")
spark.table("workspace.default.gold_attrition_metrics").display()

print("gold_attrition_by_department")
spark.table("workspace.default.gold_attrition_by_department").display()

print("gold_attrition_by_job_role")
spark.table("workspace.default.gold_attrition_by_job_role").display()
```

- verificar o resultado de cada uma

- Resultado da Tabela — gold_attrition_metrics:

gold_attrition_metrics						
Table						
	total_employees	total_attrition	attrition_rate	avg_monthly_income	avg_age	avg_years_at_company
1	1470	237	0.16122448979591836	6502.931292517007	36.9238095238095...	7.0081632653061225

Dados tabulados:

Métrica	Valor
total_employees	1470
total_attrition	237
attrition_rate	0.1612 (16.12%)
avg_monthly_income	6502.93
avg_age	36.92
avg_years_at_company	7.01
percent_overtime	0.2830 (28.30%)
avg_job_satisfaction	2.73
avg_work_life_balance	2.76

Interpretação do resultado: A empresa apresenta uma taxa de turnover de 16.12%, chama atenção, pois pode não ser ideal para ambientes saudáveis.

Apesar de uma boa média salarial e colaboradores experientes (37 anos, 7 anos de casa), há sinais de preocupação:

- 28% fazem hora extra → risco de burnout
- Satisfação mediana (2.73) → clima organizacional moderado
- Equilíbrio vida-trabalho mediano (2.76) → possível sobrecarga
- 237 desligamentos no período → impacto financeiro e operacional

Em RH, existem padrões amplamente documentados, onde:

- alta carga de trabalho → burnout → desligamento
- baixa satisfação → desligamento
- baixo equilíbrio vida-trabalho → desligamento
- problemas de gestão → desligamento

- falta de carreira → desligamento

Desta forma, o banco de dados na etapa Gold está preparado para análises futuras do departamento de Recursos Humanos.

- Resultado da Tabela — gold_attrition_by_department:

gold_attrition_by_department

department	total_employees	total_attrition	attrition_rate	avg_monthly_income	avg_age	avg_job_satisf
1 research & developme...	961	133	0.1383975026014568	6281.252861602497	37.0426638917794	2.726326
2 sales	446	92	0.2062780269058296	6959.17264573991	36.54260089686099	2.75112
3 human resources	63	12	0.19047619047619047	6654.507936507936	37.80952380952381	2.60317

Dados tabulados:

Departamento	Total de Funcionários	Total de Desligamentos	Taxa de Attrition	Renda Média Mensal	Idade Média	Satisfação Média	Anos Médios na Empresa
Research & Development	961	133	0.1384 (13.84%)	6281.25	37.04	2.73	6.86
Sales	446	92	0.2063 (20.63%)	6959.17	36.54	2.75	7.28
Human Resources	63	12	0.1905 (19.05%)	6654.51	37.81	2.60	7.24

Interpretação do resultado gold_attrition_by_department:

A análise por departamento mostra que o Research & Development possui a menor taxa de attrition (13,84%), indicando uma retenção relativamente melhor, com colaboradores experientes e satisfação moderada. Já o departamento de Sales apresenta a maior taxa de desligamentos (20,63%), sugerindo maior pressão operacional ou desafios específicos da área comercial, mesmo com a maior renda média entre os departamentos. O setor de Human Resources também registra uma taxa elevada (19,05%), possivelmente refletindo sobrecarga administrativa ou menor satisfação interna. No geral, a rotatividade é mais intensa em Sales e Human Resources, enquanto Research & Development demonstra maior estabilidade, embora ainda com espaço para melhorias.

Esses pontos geram preocupação e o resultado abre frente para futuras análises de Recursos humanos e verificar as correlações possíveis.

- Resultado da Tabela — gold_attrition_by_job_role:

3 rows3.847s runtime

Refreshed 38m ago

gold_attrition_by_job_role

Table

Count 63

	1.2 total_employees	1.2 total_attrition	1.2 attrition_rate	1.2 avg_monthly_income	1.2 avg_years_in_role	1.2 avg_job_satisfaction
2	131	9	0.06870229007633588	7528.763358778626	4.862595419847328	2.786259541984733
3	52	12	0.23076923076923078	4235.75	3.1346153846153846	2.5576923076923075
4	326	57	0.17484662576687116	6924.2791411042945	4.855828220858895	2.754601226993865
5	145	10	0.06896551724137931	7295.137931034483	4.972413793103448	2.682758620689655
6	292	47	0.16095890410958905	3239.972602739726	3.2739726027397262	2.7739726027397262
7	83	33	0.39759036144578314	2626	2.0120481927710845	2.7349397590361444
8	259	62	0.23938223938223938	3237.169884169884	3.200772200772201	2.6911196911196913
9	102	5	0.049019607843137254	17181.676470588234	6.450980392156863	2.7058823529411766

9 rows3.847s runtime

Refreshed 38m ago

Dados tabulados:

Cargo (job_role)	Total de Funcionários	Total de Desligamentos	Taxa de Attrition	Renda Média Mensal	Anos Médios no Cargo	Satisfação Média
Research Director	80	2	0.025 (2.5%)	16033.55	6.29	2.70
Healthcare Representative	131	9	0.0687 (6.87%)	7528.76	4.86	2.79
Human Resources	52	12	0.2308 (23.08%)	4235.75	3.13	2.56
Sales Executive	326	57	0.1748 (17.48%)	6924.28	4.86	2.75
Manufacturing Director	145	10	0.0690 (6.90%)	7295.14	4.97	2.68
Research Scientist	292	47	0.1609 (16.09%)	3239.97	3.27	2.77
Sales Representative	83	33	0.3976 (39.76%)	2626.00	2.01	2.73
Laboratory Technician	259	62	0.2394 (23.94%)	3237.17	3.20	2.69
Manager	102	5	0.0490 (4.90%)	17181.68	6.45	2.71

Interpretação do Resultado da gold_attrition_by_job_role:

A análise por cargo revela diferenças marcantes na rotatividade entre as funções da empresa.

Os cargos de liderança, como Research Director e Manager, apresentam as menores taxas de attrition, 2,5% e 4,9%, respectivamente, indicando alta estabilidade, maior remuneração e tempo médio elevado na função, fatores que normalmente contribuem para retenção.

Em contraste, funções operacionais e de entrada, como Sales Representative e Laboratory Technician, exibem as maiores taxas de desligamento, 39,76% e 23,94%,

sugerindo maior pressão, menor remuneração e menor tempo médio no cargo, o que tende a aumentar a rotatividade.

Cargos técnicos como Research Scientist também apresentam uma taxa elevada (16,09%), possivelmente refletindo desafios de carreira ou carga de trabalho.

Já funções intermediárias, como Sales Executive, mostram uma taxa significativa (17,48%), alinhada ao comportamento observado no departamento de vendas.

No geral, a rotatividade é mais intensa em cargos operacionais e comerciais, enquanto posições de liderança e alta especialização demonstram maior estabilidade.

Esses pontos geram preocupação e o resultado abre frente para futuras análises de Recursos humanos e verificar as correlações possíveis.

Próxima etapa do Gold: Modelagem dos Dados

Para a modelagem analítica, os dados foram estruturados em um esquema estrela, no qual uma tabela fato central reúne os indicadores relacionados à permanência e ao desligamento dos funcionários, enquanto tabelas dimensão armazenam atributos descritivos utilizados para segmentação e comparação. Essa organização facilita consultas, criação de métricas e análises por diferentes perspectivas, mantendo relacionamentos simples e consistentes entre os dados.

O esquema estrela implementado é composto pelos seguintes elementos:

Tabela Fato — fact_attrition: contém uma linha por funcionário, reunindo medidas numéricas relevantes às análises, como indicação de desligamento, idade, renda mensal, anos de empresa, satisfação no trabalho, equilíbrio vida-trabalho e horas extras. A tabela também armazena as chaves substitutas que conectam cada registro às dimensões de departamento e cargo.

Dimensão Departamento — dim_department: armazena os departamentos existentes na empresa, permitindo segmentar a rotatividade por áreas organizacionais.

Dimensão Cargo — dim_job_role: contém os cargos e funções desempenhadas pelos colaboradores, possibilitando análises comparativas entre diferentes posições.

Relacionamentos: cada dimensão se conecta à tabela fato por meio de chaves substitutas, estabelecendo relacionamentos de um para muitos. Essa estrutura evita duplicidade, melhora a performance das consultas e facilita a criação de filtros combinados em ferramentas de BI.

Preparação dos dados: antes da criação das tabelas, foram realizadas validações de tipos, padronização de categorias, seleção dos atributos relevantes e carga das dimensões, seguida da construção da tabela fato com as junções apropriadas.

Esse modelo estrela fornece uma base analítica sólida para explorar a rotatividade por diferentes perspectivas, permitindo análises rápidas, consistentes e escaláveis em ambientes como Databricks SQL e Power BI.

Criação das tabelas no databricks da modelagem *Star Schema*:

- criar dim_department

3 minutes ago (6s)6Python

Bloco 6 - Criar dim_department

from pyspark.sql import functions as F

df_dim_department = (
 df_silver
 .select("department")
 .distinct()
 .withColumn("department_id", F.monotonically_increasing_id())
)

df_dim_department.write.format("delta").mode("overwrite").saveAsTable("workspace.default.dim_department")

spark.table("workspace.default.dim_department").display()

> See performance (2)✓ Checked for improvements

> df_dim_department: pyspark.sql.connect.dataframe.DataFrame = [department: string, department_id: long]

Table +

Q 🔍 ⌵ ⌵

	department	department_id
1	research & developme...	0
2	sales	1
3	human resources	2

- Criar dim_job_role

Just now (5s)7Python

Bloco 7 - Criar dim_job_role

df_dim_job_role = (
 df_silver
 .select("job_role")
 .distinct()
 .withColumn("job_role_id", F.monotonically_increasing_id())
)

df_dim_job_role.write.format("delta").mode("overwrite").saveAsTable("workspace.default.dim_job_role")

spark.table("workspace.default.dim_job_role").display()

> See performance (2)✓ Checked for improvements

> df_dim_job_role: pyspark.sql.connect.dataframe.DataFrame = [job_role: string, job_role_id: long]

Resultado do dim_job_role:

> See performance (2)

> df_dim_job_role: pyspark.sql.connect.dataframe.DataFrame = [job_role: string, job_role_id: long]

Table +

⌵ ⌵ ⌵ ⌵ ⌵ ⌵ ⌵ ⌵ ⌵ ⌵

	job_role	job_role_id
1	Research Director	0
2	Healthcare Representative	1
3	Human Resources	2
4	Sales Executive	3
5	Manufacturing Director	4
6	Research Scientist	5
7	Sales Representative	6
8	Laboratory Technician	7
9	Manager	8

9 rows 5.370s runtime

- Criar fact_attrition

```
Just now (7s) 8 Python

# Carregar dimensões
dim_dept = spark.table("workspace.default.dim_department")
dim_role = spark.table("workspace.default.dim_job_role")

# Criar tabela fato
df_fact_attrition = (
    df_silver
        .join(dim_dept, "department", "left")
        .join(dim_role, "job_role", "left")
        .select(
            "employee_number", # identificador do funcionário no dataset original
            "attrition",
            "age",
            "monthly_income",
            "years_at_company",
            "job_satisfaction",
            "work_life_balance",
            "over_time",
            "department_id",
            "job_role_id"
        )
)

df_fact_attrition.write.format("delta").mode("overwrite").saveAsTable("workspace.default.fact_attrition")

spark.table("workspace.default.fact_attrition").display()

> See performance \(2\) ✓ Checked for improvements
```

Resultado:

> [See performance \(2\)](#) ✓ Checked for improvements

> 3 dataframes: df_fact_attrition, dim_dept, dim_role

	employee_number	attrition	age	monthly_income	years_at_company	job_satisfaction	work_life_balance	over_time
1	1	true	41	5993		6	4	1 true
2	2	false	49	5130		10	2	3 false
3	4	true	37	2090		0	3	3 true
4	5	false	33	2909		8	3	3 true
5	7	false	27	3468		2	2	3 false
6	8	false	32	3068		7	4	2 false
7	10	false	59	2670		1	1	2 true
8	11	false	30	2693		1	3	3 false
9	12	false	38	9526		9	3	3 false
10	13	false	36	5237		7	3	2 false
11	14	false	35	2426		5	2	3 false
12	15	false	29	4193		9	3	3 true
13	16	false	31	2911		5	3	2 false
14	18	false	34	2661		2	4	3 false

15

1,470 rows 6.693s runtime Refreshed Now

- Validação da Star Schema

```
3 minutes ago (4s) 9 Python

# Bloco 10 - Validar Star Schema

print("dim_department")
spark.table("workspace.default.dim_department").display()

print("dim_job_role")
spark.table("workspace.default.dim_job_role").display()

print("fact_attrition")
spark.table("workspace.default.fact_attrition").display()

print("Verificar se todos os registros da fato possuem departamento e cargo válidos:")
spark.sql("""
SELECT
    COUNT(*) AS total_registros,
    SUM(CASE WHEN department_id IS NULL THEN 1 ELSE 0 END) AS dept_null,
    SUM(CASE WHEN job_role_id IS NULL THEN 1 ELSE 0 END) AS role_null
FROM workspace.default.fact_attrition
""").display()

> See performance \(4\) ✓ Checked for improvements
```

- Resultados:

dim_department

Table  

	^A _C department	¹ ² ₃ department_id
1	research & developme...	0
2	sales	1
3	human resources	2

dim_job_role

Table  

	^A _C job_role	¹ ² ₃ job_role_id
1	Research Director	0
2	Healthcare Representative	1
3	Human Resources	2
4	Sales Executive	3
5	Manufacturing Director	4
6	Research Scientist	5
7	Sales Representative	6
8	Laboratory Technician	7
9	Manager	8

  9 rows | 4.485s runtime

fact_attrition

Table  

	¹ ² ₃ employee_number	¹ ² ₃ attrition	¹ ² ₃ age	¹ ² ₃ monthly_income	¹ ² ₃ years_at_company	¹ ² ₃ job_satisfaction	¹ ² ₃ work_life_balance	¹ ² ₃ over_time
1	1	true	41	5993	6	4	1	true
2	2	false	49	5130	10	2	3	false
3	4	true	37	2090	0	3	3	true
4	5	false	33	2909	8	3	3	true
5	7	false	27	3468	2	2	3	false
6	8	false	32	3068	7	4	2	false
7	10	false	59	2670	1	1	2	true
8	11	false	30	2693	1	3	3	false
9	12	false	38	9526	9	3	3	false
10	13	false	36	5237	7	3	2	false
11	14	false	35	2426	5	2	3	false
12	15	false	29	4193	9	3	3	true
13	16	false	31	2911	5	3	2	false
14	18	false	34	2661	2	4	3	false
15								

  1,470 rows | 4.485s runtime

Refreshed 5m ago

Verificando registro da fact se possuem departamento e cargos válidos:

	1.2 total_registros	1.3 dept_null	1.3 role_null
1	1470	0	0

1 row | 4.485s runtime

Próxima etapa: verificando integração com SQL Editor

Teste: da taxa_attrition:

The screenshot shows the Databricks SQL Editor interface. On the left is a sidebar with navigation options like Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL Editor (selected), Queries, Dashboards, Genie Agents, Alerts, Query History, and SQL Warehouses. The main area displays a SQL query in the editor:

```

1 SELECT
2   COUNT(*) AS total_funcionarios,
3   SUM(CASE WHEN attrition = TRUE THEN 1 ELSE 0 END) AS total_desligados,
4   SUM(CASE WHEN attrition = TRUE THEN 1 ELSE 0 END) / COUNT(*) AS taxa_attrition
5 FROM workspace.default.fact_attrition;
6
7

```

Below the query editor, the results of the query are shown in a table:

	1.2 total_funcionarios	1.3 total_desligados	1.2 taxa_attrition
1	1470	237	0.16122448979591836

Após a checagem, o databricks está configurado para futuras consultas adicionais por SQL pelo departamento de Recursos Humanos.

QUALIDADE DE DADOS

Durante a construção das camadas e do modelo analítico, aplicamos um conjunto de práticas essenciais de **Data Quality**, garantindo que os dados estivessem limpos, consistentes e prontos para análises confiáveis.

1. Padronização de tipos (Silver Layer)

Convertemos colunas para os tipos corretos, garantindo que atributos como idade, renda, anos de empresa e indicadores booleanos fossem interpretados corretamente pelo Spark e pelo Databricks SQL.

2. Limpeza e padronização de categorias

Normalizamos valores textuais como department, job_role, over_time e attrition, assegurando consistência para criação das dimensões e para evitar duplicidades.

3. Garantia de unicidade nas dimensões

As dimensões foram criadas com `.distinct()`, assegurando que cada departamento e cada cargo aparecessem apenas uma vez — requisito fundamental de qualidade em modelos estrela.

4. Verificação de integridade referencial

Foi validado se a tabela fato estava corretamente ligada às dimensões, verificando chaves nulas e possíveis registros órfãos.

5. Conversão de booleanos para métricas numéricas no SQL

Ao rodar consultas no Databricks SQL, foi tratado o atributo `attrition` (booleano) com `CASE WHEN` para permitir cálculos como soma e média — garantindo consistência analítica.

Com essas etapas, garantimos que:

- os dados estão consistentes,
- as dimensões são confiáveis,
- a tabela fato está íntegra,
- as métricas podem ser calculadas sem distorções,
- o modelo estrela está pronto para BI e análises avançadas.

CONCLUSÃO

Com base nas etapas realizadas ao longo do projeto, é possível concluir que o processo de construção do pipeline de dados, da modelagem analítica e das consultas exploratórias permitiu estruturar um ambiente confiável e eficiente para análise de rotatividade de funcionários. A organização dos dados em camadas (Bronze, Silver e Gold), aliada às práticas de qualidade aplicadas, garantiu consistência, padronização e integridade das informações utilizadas nas análises.

A modelagem em esquema estrela demonstrou-se adequada para o contexto, permitindo consultas rápidas, métricas claras e segmentações flexíveis. A tabela fato consolidou indicadores essenciais — como idade, renda, anos de empresa, satisfação, equilíbrio vida-trabalho e ocorrência de desligamento — enquanto as dimensões de departamento e cargo possibilitaram análises comparativas entre áreas e funções da organização.

As verificações de qualidade de dados realizadas ao longo do pipeline asseguraram que os atributos estivessem padronizados, sem duplicidades nas dimensões e com integridade referencial preservada. Além disso, ajustes como a conversão de valores booleanos para métricas numéricas no Databricks SQL permitiram calcular indicadores de forma precisa, como taxa geral de `attrition` e taxas segmentadas por departamento e cargo.

A partir das consultas SQL executadas, observou-se que o ambiente está preparado para análises mais profundas, construção de dashboards em ferramentas de BI e até evolução para modelos preditivos de risco de desligamento. Assim, o trabalho realizado estabelece uma base sólida para que a empresa compreenda melhor seus padrões de

rotatividade, identifique áreas críticas e apoie decisões estratégicas relacionadas à gestão de pessoas.

TODAS AS PERGUNTAS FORAM RESPONDIDAS?

Ao longo do desenvolvimento do modelo analítico e das consultas SQL, foi possível responder parte das perguntas gerais sobre rotatividade, enquanto outras permaneceram em aberto devido à ausência de determinados atributos na camada Gold. A taxa geral de rotatividade pôde ser calculada com precisão, assim como a identificação das áreas e cargos que concentram os maiores índices de desligamento, já que as dimensões de departamento e cargo foram devidamente estruturadas. Também foi possível analisar fatores como horas extras, satisfação no trabalho e equilíbrio entre vida pessoal e profissional, pois esses atributos foram incluídos na tabela fato e permitiram comparações diretas entre funcionários desligados e retidos. Além disso, diferenças relacionadas à idade, renda mensal e tempo de empresa puderam ser avaliadas, já que esses campos estavam presentes no modelo.

Por outro lado, algumas perguntas não puderam ser respondidas com o conjunto atual de dados. Informações como distância entre residência e trabalho, nível de escolaridade, gênero e frequência de viagens a trabalho não foram incluídas na camada Gold, o que impossibilitou análises relacionadas a esses fatores. Assim, perguntas sobre variação da distância por cargo, impacto de viagens a trabalho na rotatividade, diferenças por escolaridade e comparações entre gêneros permaneceram sem resposta. Para que essas análises sejam possíveis, seria necessário reprocessar a camada Silver e atualizar a Gold, incorporando esses atributos adicionais que será feito em outra oportunidade.